

Merging Screen Space Ambient Occlusion Techniques

Berten Xander

Graduation work 2024-2025



Contents

Abstract & Keywords	3
Preface	4
List of figures	4
Introduction	6
Literature study / Theoretical framework	8
1 What is SSAO?	8
1.1 SSAO Workflow	9
1.2 Mathematical Representation of SSAO	13
1.3 Noticeable SSAO techniques	15
2 Compute Shaders	16
2.1 Advantages of Compute Shaders	16
2.2 Downsides of Compute Shaders	17
Research	18
3 Framework And Tools	19
3.1 The Forge	19
3.2 Nvidia Nsight Graphics	19
3.3 RenderDoc	19
4 Testing Setup	20
4.1 Performance	20
4.2 Visual	21
5 Fast Modern SSAO	22
5.1 Implementation Details	22
5.2 Results	23

6	Ground Truth Ambient Occlusion	26
6.1	Implementation Details	26
6.2	Results	27
7	Merged Technique	29
7.1	Implementation Details	29
7.2	Results	30
	Discussion	32
	Conclusion	33
	Future work	34
	Critical Reflection	35
	References	36

Abstract & Keywords

Screen Space Ambient Occlusion (SSAO) enhances the realism of a real time rendered scene's by simulating the soft shadows created by occluded ambient light. This research explores the potential of combining traditional SSAO techniques with compute shader optimization and downscaling to improve performance. A method is proposed that integrates downscaled AO calculation, ground truth sampling, achieving significant speed improvement over the traditional ground truth sampling.

The study evaluates existing SSAO variants, such as Ground Truth Ambient Occlusion (GTAO), and a fast downscaled Screen Space Ambient Occlusion (SSAO), to craft a hybrid approach with compute shaders.

Keywords:

Screen Space Ambient Occlusion (SSAO), Compute Shaders, Ground Truth Based Sampling, Downscaled AO, Real-Time Rendering, Graphics Optimization, Post-Processing, Ambient Occlusion Techniques

Preface

I have a deep interest in the challenges of simulating the physics of lighting in real-time rendering. The process of creating efficient yet somewhat physically correct solutions in this field feels like solving a complex puzzle, where every decision has the right balance between efficiency and visual correctness.

Among the numerous techniques in rendering, Screen Space Ambient Occlusion (SSAO) stood out to me.

The variety of SSAO implementations each with its own strengths and limitations are almost endless, this exploration started a rabbit hole trying to understand not just the mechanics of SSAO, but also the broader principles of rendering. SSAO plays a crucial role in modern games.

It simulates how light interacts with surfaces and objects in proximity, creating subtle shadowing effects within the scene. Without SSAO, games often feel flat or detached, lacking the visual cues that make environments immersive.

By adding these ambient shadows, SSAO achieves a more believable render while maintaining real-time.

This explains my journey of investigation, experimentation, and discovery. I wanted to understand the nuances of existing approaches by studying and implementing these methods

List of Figures

1	An SSAO example [13]	8
2	An example of a depth and normal buffer.	9
3	A Vogel Disk Pattern	10
4	SSAO calculation. [1]	11
5	Comparison of various SSAO techniques.	15
6	Renderdoc and Nvidia Nsight	19
7	Render of San Miguel	21
8	Comparison of the Fast SSAO	24
9	A "slice" in the hemisphere.	26
10	Comparison of GTAO	28
11	Comparison of the Merged Technique	31

Introduction

Screen Space Ambient Occlusion (SSAO) is a method used in video games and 3D graphics to make lighting look more realistic. It simulates Global Illumination at a fraction of its cost.

It helps show how light interacts with small spaces like cracks, corners, or crevices, making the scene feel more three-dimensional.

SSAO was first introduced by Crytek in the game Crysis in 2007 [10], and it was a big step forward because it worked faster than older methods. However, even today, SSAO can sometimes slow down games or look grainy, showing that there's still work to be done to make it better.

Making games look realistic has always been a big goal for graphics programmers. Lighting and shadows are especially important for making scenes feel real. Ambient occlusion is when light gets blocked in certain areas, like where two walls meet or under objects.

In the past, it took a lot of computer power to calculate this effect, so it was mostly used in movies.

SSAO changed this by finding a way to create similar effects quickly enough to use in video games. It does this by using data from the screen, like the distance and angles of objects, to guess where light would be blocked. This made games and virtual worlds look way more realistic without slowing down the calculations too much.

SSAO is a clever technique, but it's not perfect. It only works with the part of the scene the player can see, so it can miss things outside the camera's view. This can lead to mistakes or weird-looking shadows.

Early versions of SSAO also had problems with noise, which made shadows look speckled or shaky. To fix this, developers added steps like blurring to smooth out the image, but these extra steps can slow things down, making

it harder to keep games running smoothly.

Even with its issues, SSAO is still widely used because it makes a big difference in how scenes look. It helps create a sense of depth and makes virtual worlds feel more alive.

However, there's still a lot of room to make SSAO better. Researchers are working on ways to improve how SSAO works, like finding smarter ways to sample data or reducing the need for extra processing steps.

Some are even combining SSAO with other techniques to get better results. While SSAO has been around for a while, its ongoing use shows how important it is to creating realistic graphics.

This study focuses on understanding and improving SSAO because many games today still struggle with its performance and quality.

The main questions are:

What are the differences between a fast SSAO and a more realistic SSAO like GTAO.

Could we merge these SSAO techniques to use less bandwidth and be faster?

What will be the pros and cons of doing that.

The hypothesis of this research is:

H1: GTAO will be radiometrically way more correct. Thus, be visually more appealing, while it will be considerably slower.

H2: The merged technique will be way faster, it will lose a lot of its visually appealing factor. But still be visually better than the fast implementation. I suspect it to at least be 25% slower than the fast technique.

Literature study / Theoretical framework

1 What is SSAO?

Screen Space Ambient Occlusion (SSAO) is a graphics technique used to simulate the soft shadowing effects caused by ambient light being occluded in corners, crevices, and near surfaces in a 3D scene.

It is simulating the Ambient Occlusion of Global Illumination. Which is the unobtainable “holy grail” of real time graphics.

It does that by leveraging information from the depth buffer and screen-space normals, SSAO provides a relatively low-cost way to enhance realism in rendered scenes.

Unlike traditional ambient occlusion (AO) methods that operate on the entire 3D scene geometry, SSAO is a post-processing effect. It calculates occlusion after rendering the scene using only the visible portions within the camera’s view (hence, “screen space”). This trade-off provides efficiency at the expense of some accuracy.

Even though by many people it is seen as a post process effect, it should be applied in the Indirect lighting And not as a post process effect. This is a common misconception about Screen Space Ambient Occlusion

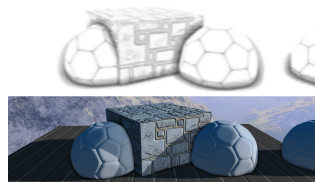


Figure 1: An SSAO example [13]

1.1 SSAO Workflow

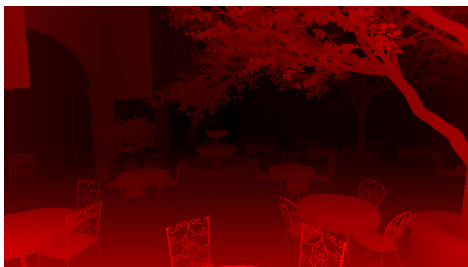
Screen Space Ambient Occlusion (SSAO) have many flavors and variations, but all of them boil down to these basic steps. [3, 4, 6, 9, 11, 13, 18]

Depth and Normal extraction

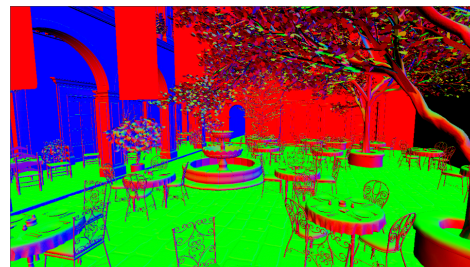
Before any ambient occlusion calculations can be performed, we need to extract critical information about the geometry of the scene:

- **Depth Buffer:** This texture stores the distance from the camera to each pixel in the scene. It helps determine how far away an object is from the viewer and is used to calculate spatial relationship between pixels.
- **Normal Buffer:** This texture stores the surface normal at each pixel. Surface normals are vectors perpendicular to the surface, providing information about the orientation of the surface. They are essential for determining how much a surface is occluded based on its angle relative to neighboring surfaces. Note that the normal buffer will be transformed to view space once it is being used.

Once generated, these buffers serve as the foundation for the SSAO algorithm.



(a) Depth Buffer



(b) Normal Buffer

Figure 2: An example of a depth and normal buffer.

Sampling and Occlusion calculation

This is the core step of the SSAO algorithm, where we approximate the occlusion for each pixel by analyzing its surrounding pixels.

- **Sampling:**

Sampling involves picking a number of points (or rays) around each pixel. These samples are distributed within a hemisphere oriented along the pixel's surface normal, as the occlusion only depends on objects above the surface (not below).

Screen-Space Sampling: Instead of sampling in 3D world space, SSAO operates in 2D screen space using the depth buffer to reconstruct approximate 3D positions of points. This is computationally efficient but introduces some inaccuracies.

The samples are typically jittered or randomized to reduce noticeable patterns, using techniques such as:

- **Random Rotations:** Rotate the sampling kernel for each pixel to reduce uniformity.
- **Spiral Sampling Patterns:** Use deterministic sampling distributions like Poisson disk sampling to ensure good coverage of the hemisphere.

A Pattern and Rotation that is commonly used with shadow techniques is called Vogel Disk or Vogel Spiral. It is a fairly simple and fast algorithm where samples get rotated, will be spread out evenly and will never map onto themselves, unless you rotate it by 2π .

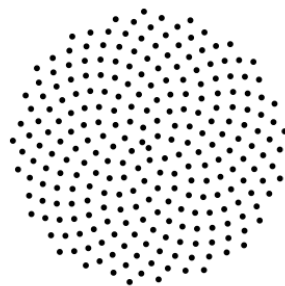


Figure 3: A Vogel Disk Pattern

- **Occlusion Calculation:**

For each sample, the algorithm computes how much the sampled point occludes the pixel being processed.

This is typically done by:

Reconstructing the view-space position of the sampled point using the depth buffer. Comparing the depth of the sampled point to the current pixel.

If the sampled point is closer than expected based on its distance from the current pixel, it contributes to occlusion.

Using the surface normal of the current pixel to weigh the contribution of each sample, as surfaces facing away from the sampled point are less likely to be occluded.

The occlusion value is aggregated for all samples and normalized.

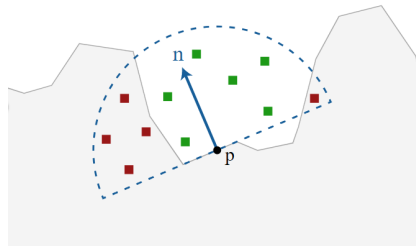


Figure 4: SSAO calculation. [1]

Blur/Noise Reduction

Since SSAO relies on sampling, it often produces noise or visual artifacts, particularly when the number of samples is low (to maintain performance). To address this, a blur or filter is applied to smooth the occlusion map.

- **Bilateral Blur:** A bilateral blur is commonly used because it respects edges in the image. This means it smooths out noise while preserving details like object boundaries. This is achieved by weighting the blur based on both spatial proximity (distance between pixels) and similarity in depth or normals.
- **Gaussian Blur:** A simpler Gaussian blur can also be applied. However, it might cause bleeding across object boundaries, resulting in incorrect occlusion values near edges. This can be resolved by also taking the depth into account while blurring the texture.
- **Temporal Filtering:** In addition to spatial blurring, temporal filtering can be applied by averaging occlusion values over multiple frames, further reducing noise in dynamic scenes. The result of this step is a smooth, visually appealing ambient occlusion map.

Integration

The final step involves combining the computed ambient occlusion map with the scene's lighting to produce the final rendered image.

- **Modulating Ambient Light:** Ambient occlusion affects only the ambient lighting component of the scene. This means the computed AO map is multiplied with the ambient light intensity to darken occluded areas while leaving directly lit areas unchanged.
- **Applying AO in Shading:** Once the AO map is computed, it is blended with the existing shading calculations.
- **Tone Mapping:** After integration, the final image undergoes tone mapping and color correction to produce the final frame. The AO effect is subtle but adds significant depth and realism to the scene.

1.2 Mathematical Representation of SSAO

We can split the lighting calculations into direct lighting and indirect lighting components. As you can see, this resembles the general rendering equation. The indirect lighting calculation can be written as:

$$\int_{\Omega} L_{\text{ambient}}(\omega_i) \underbrace{\cos \theta_i}_{\text{Angle of Incoming Light}} \underbrace{f_r(\omega_i, \omega_o)}_{\text{Scattering}} d\omega_i$$

To simplify this, we approximate the scattering part of the equation $f_r(\omega_i, \omega_o)$ with a constant value K_a , which represents the material's ambient reflectance. This leads to:

$$L_a K_a \int_{\Omega} L_{\text{ambient}}(\omega_i) \cos \theta_i d\omega_i$$

Next, we introduce a visibility function: $V(\omega_i)$ which determines whether a point is occluded from a particular direction. When $V(\omega_i) = 1$, the integral evaluates to π , which corresponds to the sum of all cosines over the hemisphere. Dividing by π normalizes this result to a value between 0 and 1, which is what we want. This gives:

$$L_a K_a \frac{1}{\pi} \int_{\Omega} \underbrace{V(\omega_i) \cos \theta_i}_{\text{Ambient Occlusion}} d\omega_i$$

To compute ambient occlusion for a specific pixel, we need the normal vector $\hat{n}$ at that position. This modifies the equation to:

$$\text{AO}_{p,\hat{n}} = \frac{1}{\pi} \int_{\Omega} V_{p,\hat{\omega}}(\hat{n} \cdot \hat{\omega}) d\hat{\omega}$$

Now that we have our Ambient Occlusion Equation we want to simplify and approximate it so that a GPU can do in real time. We will start with Monte Carlo Sampling: this is a technique used to approximate complex mathematical operations, like integrals, by randomly sampling points and averaging their contributions.

$$\text{AO}_{p,\hat{n}} \approx \frac{1}{\pi} \cdot \frac{1}{s} \sum_{i=1}^s V_{p,\hat{\omega}_i}(\hat{n} \cdot \hat{\omega}_i)$$

For practical computation, we absorb the normalization factor $\frac{1}{\pi}$ into the visibility function:

$V_{p,\hat{\omega}_i}(\hat{n} \cdot \hat{\omega}_i)$. Since the directions ω_i are sampled as unit vectors during Monte Carlo sampling, their magnitude is inherently 1. With this adjustment, the equation simplifies to:

$$\text{AO}_{p,\hat{n}} \approx \frac{1}{s} \sum_{i=1}^s V_{p,\hat{\omega}_i}(\hat{n} \cdot \hat{\omega}_i)$$

Now that we have simplified the ambient occlusion equation into a summation, the next step is to approximate the visibility function $V_{p,\hat{\omega}_i}$. This function determines whether a point is occluded from a particular direction ω_i .

Instead of computing visibility directly, which involves expensive ray-tracing operations, we approximate it using a heuristic. We assume the visibility is influenced by the depth or distance between the sample point p_i and the current pixel p . This gives us a simple expression for $V_{p,\hat{\omega}_i}$

$$V_{p,\hat{\omega}_i} \approx \frac{\max(0, \vec{v}_i \cdot \hat{n} + p_{z_c}\beta)}{\vec{v}_i \cdot \vec{v}_i + \epsilon}$$

- $\vec{v}_i = p_i - p$: Represents the vector from the current pixel p to the sampled point p_i .
- p_{z_c} : Represents a depth bias constant to prevent over-occlusion.
- β : Represents a constant to control the influence of the depth term.
- ϵ : Represents a small constant to avoid division by zero.

This approximation not only simplifies the visibility calculation but also ensures numerical stability, making it suitable for real-time GPU rendering. Substituting this visibility approximation back into the summation, the ambient occlusion equation becomes:

$$\text{AO}_{p,\hat{n}} \approx \frac{1}{s} \sum_{i=1}^s \frac{\max(0, \vec{v}_i \cdot \hat{n} + p_{z_c}\beta)}{\vec{v}_i \cdot \vec{v}_i + \epsilon}, \quad \vec{v}_i = p_i - p$$

Now this approximation is relatively fast and good enough to calculate Ambient Occlusion in real time

1.3 Noticeable SSAO techniques

Over the years, SSAO techniques have evolved to improve visual fidelity, reduce noise, and optimize performance. Below is an overview of major advancements:

Technique	Introduced By	Year	Improvement
SSAO	Crytek	2007	Baseline implementation. Introduced the concept of screen-space occlusion using depth buffer sampling.
HBAO	NVIDIA	2008	Added horizon-based calculations for improved accuracy.
HDAO	AMD	2009	Optimized for AMD GPUs. Enhanced precision and performance.
SSDO	Ubisoft	2010	Introduced directional light and color bleeding effects.
HBAO+	NVIDIA	2013	Optimized HBAO with fewer artifacts and lower overhead.
GTAO	Activision	2016	Better quality with the same number of depth buffer samples.
VXAO	NVIDIA	2016	Extended to world-space using voxelization for high-quality occlusion.
ASSAO	Intel	2019	Adaptive resolution scaling for performance optimization.
RTAO	NVIDIA	2020s	Ray-traced AO for physically accurate results.
NNAO	Various	2020s	Neural network-based AO for high accuracy and speed.

Table 1: Screen-space ambient occlusion (SSAO) techniques and improvements.



Figure 5: Comparison of various SSAO techniques.

2 Compute Shaders

The growing use of compute shaders has revolutionized real-time graphics by offering a flexible and efficient pipeline. Compute shaders allow developers to exploit parallelism and optimize resource utilization. For this i will also make use of Compute Shaders. [19]

2.1 Advantages of Compute Shaders

- **Bypass ROP Bottlenecks:** Unlike traditional pixel shaders, compute shaders bypass the raster operations processor (ROP) stage, which is part of the graphics pipeline. By skipping this step, compute shaders avoid potential bottlenecks and achieve better throughput, especially for tasks unrelated to rendering geometry.
- **Direct Memory Access:** Compute shaders can write directly to memory without going through additional stages like blending or frame-buffer operations. This makes them particularly efficient for tasks that require raw data processing, such as physics simulations or post-processing effects.
- **Custom Optimizations:** Compute shaders allow the developer to define thread groups explicitly. Within a thread group, intermediate results can be shared between threads using group-shared memory. This reduces redundant calculations and minimizes memory bandwidth usage.
- **Group-Shared Memory:** Compute shaders leverage group-shared memory, enabling threads within the same thread group to share sampled values or intermediate computations. This reduces the need for multiple texture fetches, further optimizing performance.
- **Synchronization:** Compute shaders provide explicit thread synchronization mechanisms, allowing developers to carefully manage thread execution and data dependencies. This level of control is not available in traditional pixel shaders.

2.2 Downsides of Compute Shaders

- **Lack of built-in features:** Pixel shaders benefit from built-in features provided by the graphics pipeline, such as:
 - Color blending
 - Anti-aliasing helper functions (e.g. `ddx`, `ddy`)
 - Automatic framebuffer handling

Compute shaders do not provide these features, so developers need to implement custom solutions for similar functionality.

- **Overhead for small tasks:** Compute shaders can be overkill for simple pixel-level operations. For effects like basic color grading or bloom, the added overhead of setting up compute dispatches and managing buffers may outweigh the benefits.
- **Undefined Behaviour with derivative functions:** While pixel shaders support derivative-based functions like `ddx`, `ddy`, and texture-level sampling seamlessly, using these in compute shaders can result in undefined behaviour. This makes compute shaders unsuitable for effects that rely heavily on derivatives (e.g., mipmapping, anisotropic filtering).

Research

In this research, I will implement and evaluate two variants of Screen Space Ambient Occlusion (SSAO). These implementations will be designed specifically to run on compute shaders. Additionally, I will investigate if i can merge the two implementations together to get a better result in the end.

To achieve these goals, i will:

- Implement Two SSAO variants
 - A general fast SSAO.
 - Ground Truth Ambient Occlusion.
- Test their performance, quality, and behaviour under various conditions, focusing on compute shaders.
- Merge the two implementations into one and see the pros and cons

The findings from this research will provide insights into the efficiency and applicability of these approaches in real-time graphics.

3 Framework And Tools

3.1 The Forge

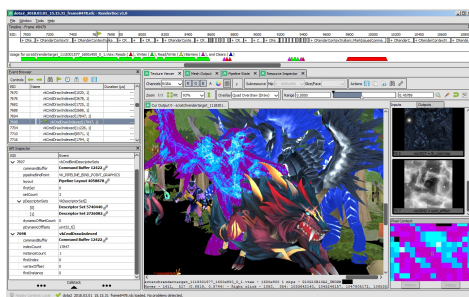
The Forge framework is a low level, multi-platform rendering framework designed to serve as a robust rendering system for game engines. It supports a wide range of platforms. It has built-in support for timing rendering passes and shader code. It also exports a nice graph of all timings and render passes with each launch of your application. This framework is more and more being used by the industry (Starfield, Call of Duty, No Man's Sky, Hypixel...)

3.2 Nvidia Nsight Graphics

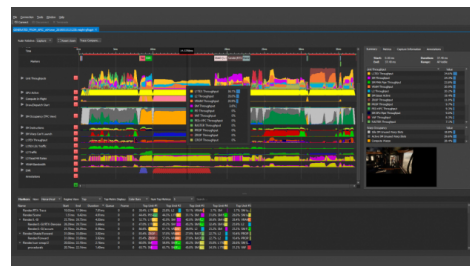
I will mainly use NVIDIA Nsight Graphics to identify bottlenecks and inefficiencies at various stages, from input assembly to final frame output. With this tool I will get detailed information about my shader code.

3.3 RenderDoc

I will use Renderdoc mostly for debugging my shaders, I can easily inspect the inputs and outputs of any command that got pushed to the GPU, this way I output some data to a texture and easily see if my shader is wrong. Also, with the nice support for Timestamps in the forge I can easily find the correct passes In this software.



(a) Renderdoc



(b) Nvidia Nsight

Figure 6: Renderdoc and Nvidia Nsight

4 Testing Setup

4.1 Performance

To evaluate performance accurately, we will render the same predefined scene using identical camera settings throughout the tests to ensure consistency and fairness.

The camera will remain stationary, and its parameters, such as position, field of view, and resolution, will be kept fixed to maintain a uniform perspective of the scene.

Performance metrics will be gathered by measuring the average frame time over the course of 120 frames, providing a reliable and stable evaluation by smoothing out any transient fluctuations or spikes in frame times.

This approach ensures a fair comparison and captures the true rendering performance under controlled conditions.

4.2 Visual

The San Miguel scene modeled by Guillermo M. Leal Llaguno will be used in these tests.

This is a complex Testing scene with big and small crevasses. Which is perfect to test the overall effectiveness of the SSAO.

I will render a ground truth image with blender using a path tracer. This will be the “physically accurate” baseline. So we can compare how well the other variants are doing visually.

Model:	San Miguel 2.0
Renderer:	Blender Cycles Path Tracer
Samples:	1024
Bounces:	4 / 12 for translucent materials
Time:	54 minutes
Camera Position:	X: -23.235 Y: -6.950 Z: 2.280
Camera Rotation:	X: 70.0° Y: 0.0° Z: 295°
Camera FOV:	90°

Table 2: Rendering settings for the San Miguel 2.0 model.

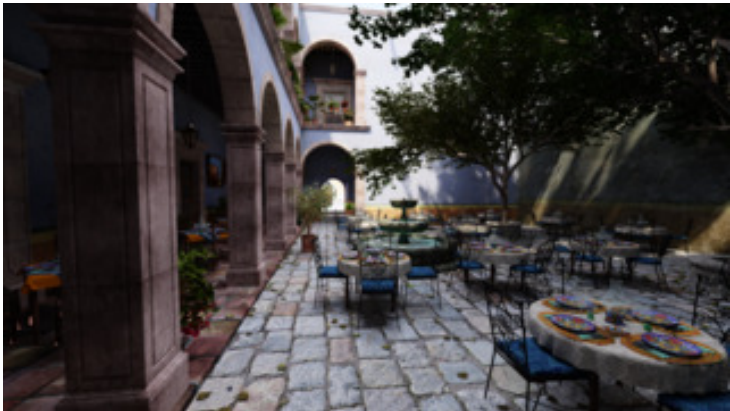


Figure 7: Render of San Miguel

5 Fast Modern SSAO

The first implementation is based on a technique described by Wojciech Sterna in GPU Zen. [14]

5.1 Implementation Details

- Downsample depth to quarter size into a R16_Float Texture. We Will pick the top right pixel for every 4 pixels and store that one in a texture. This way we end up with a depth map a quarter of the resolution of the original one.
- Use the downsampled depth and the normals from the g-buffer to calculate the Occlusion
- Do 2 depth aware blur passes We will do 2 depth aware blur passes. First, we blur over the x-axis and then the y axis It is depth aware, otherwise it could for example blur out over something that is in front of the occlusion and that will look off.
- Upsample the final blurred SSAO back to screen size.

I did use 16 samples for the ssao algorithm.
You could also get away with using 8 samples if more performance would be needed.

5.2 Results

Performance

Pass	Average Time in milliseconds
<i>Downsample Depth Pass</i>	0.0381 ms
<i>Compute SSAO Pass</i>	0.1688 ms
<i>Blur SSAO Pass</i>	0.1122 ms
<i>Upsample SSAO Pass</i>	0.08981 ms
<i>Full SSAO Calculations:</i>	0.4084 ms

Table 3: Average timings for the fast SSAO passes.

As you can see this SSAO is running under 0.5ms which is good. This variant can really fit in projects where the graphics budget is tight, Such as Mobile or Console. This SSAO implementation is efficient, running in under 0.5 milliseconds for the entire process, making it well-suited for titles with a tight Graphics Budget, such as real-time applications for mobile devices or consoles.

This SSAO variant offers a balance between visual fidelity and performance that aligns perfectly with the needs of resource-limited systems. By keeping the SSAO process so lightweight, developers can free up GPU resources for other critical elements of the rendering pipeline or improving overall frame rates.

Overall, with its sub-0.5 millisecond performance, this SSAO implementation is a perfect fit for projects where the graphics budget is tight and every millisecond counts. It enhances the feel of depth while ensuring that performance remains smooth and responsive. This makes it practical on platforms where hardware capabilities are limited.

Visual

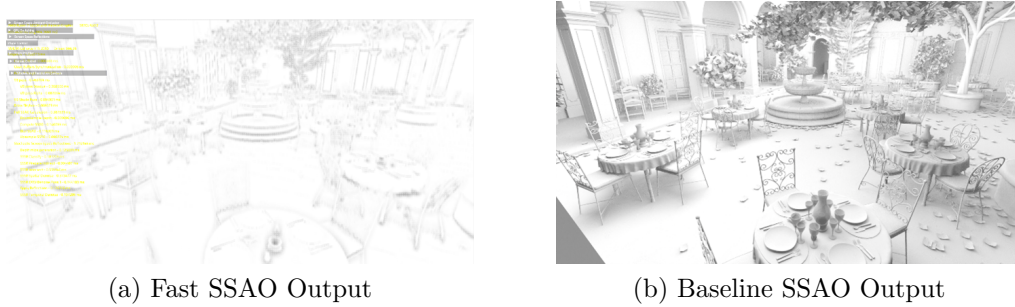


Figure 8: Comparison of the Fast SSAO

Edges

In the first image, general outlines and sharp corners are highly visible, but they appear overly pronounced. This suggests that the SSAO pass is placing too much emphasis on abrupt depth changes without accounting for smoother gradients. The second image balances the depiction of sharp edges, providing a more integrated and realistic appearance of ambient occlusion across the objects in the scene.

Thin Objects

In the first image, thin objects are unnaturally dark, this is due that the shader does not take in account the thickness of a object to simulate the physical light behaviour. This creates an unrealistic portrayal, as thin objects should allow more light to diffuse naturally. In the second image these thin elements are better integrated into the scene, with occlusion reduced to a more appropriate level. This adjustment enhances realism and aligns more closely with how light interacts with these smaller objects.

Large Occluders

Occlusion from larger objects, such as the trees, ceiling, and the central fountain, is also notably different between the two images. In the first image, large structures fail to cast realistic occlusion, leading to a flatter scene. The absence of proper shadowing undermines the illusion of depth. In the second image, large objects interact more naturally with their surroundings, casting shadows and contributing to the scene's overall depth and realism.

Contrast

The overall contrast and interaction of light within the scenes further emphasize the differences. The first image has a high-contrast appearance that suggests SSAO is being over-applied in certain areas. This results in harsh delineations between occluded and non-occluded regions, detracting from the scene's believability. On the other hand, the second image exhibits smoother gradients and more fine-tuned SSAO parameters, which simulate ambient light in a way that feels more natural and visually cohesive. Finally, when considering the global integration of the scene, the first image struggles to harmonize the interplay between larger and smaller elements. Large shadows are conspicuously missing, while smaller details are over-emphasized, creating inconsistencies. The second image addresses these issues more effectively. The occlusion appears better distributed, balancing the interaction between both large and small elements in the environment. This results in a scene that feels more cohesive and physically plausible.

6 Ground Truth Ambient Occlusion

GTAO is a more physically inspired version of SSAO. It was developed by Activision.[7]

This means that the general shader is more complex. But due to this it can generate a more accurate Ambient Occlusion with roughly the same number of samples from the depth buffer.

Which is almost always the bottleneck in Screen Space Ambient Occlusion.

6.1 Implementation Details

Ground Truth Ambient Occlusion (GTAO) stands apart from traditional SSAO by having a more physically accurate approach to ambient occlusion. It uses hemisphere slicing for sampling(splits up the hemisphere it will sample it in parts), ensuring a better coverage.

Unlike SSAO, which relies on simple depth-based approximations, GTAO applies a more correct radiological model that accounts for light attenuation, distance falloff, and angle weighting, That's way it doesn't need a ad-hoc fall-off function like most SSAO Techniques.

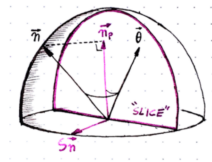


Figure 9: A "slice" in the hemisphere.

The GTAO was implemented directly into the lighting pass, allowing it to be calculated alongside the indirect lighting. This approach is the most physically accurate way to integrate SSAO, as it ensures the ambient occlusion directly interacts with the lighting in a realistic manner.

To determine the exact performance cost of the SSAO, I first had to time the entire lighting pass with GTAO included and then repeat the timing process with the SSAO component removed. This dual timing process added an extra step but was essential to isolate the impact of the SSAO on the overall frame time.

I am using 2 x 8 samples in my implementation. We can go much lower while still having a decent quality. (So this means 16 depth samples per pixel which is the same as the fast solution.).

6.2 Results

Performance

<i>Full Lighting Pass</i>	2.4341 ms
<i>Lighting Without GTAO</i>	0.8328 ms
<i>GTAO</i>	1.6013 ms

Table 4: Average timings for the GTAO calculations.

The GTAO implementation averages 1.6 ms per frame, which is surprisingly better than expected, especially considering it operates at full-screen resolution. This means a significant number of samples are being calculated, yet it still performs within an acceptable range.

That said, this time is on the upper limit of what could be considered reasonable for a screen space ambient occlusion (SSAO) pass. In the context of game performance budgets, where the total frame time should ideally not exceed 16 ms for most games targeting 60 FPS, this SSAO would consume roughly 10% of that budget. For games aiming for a higher threshold of 30 ms per frame, typical for less demanding scenarios or higher graphical fidelity at lower frame rates, the percentage drops, but it's still a substantial share.

This performance cost highlights the need to balance visual quality and computational efficiency. While GTAO demonstrates impressive performance for its complexity, it's clear that optimization opportunities or resolution scaling may be necessary for games targeting stricter frame time constraints.

Visual

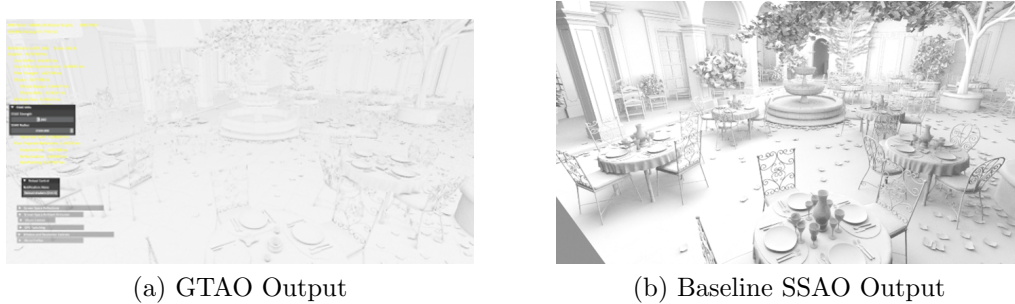


Figure 10: Comparison of GTAO

Edges

In the first image, general outlines and sharp corners are almost as pronounced as the baseline. They don't pop out as in the other implementation. It's more balanced and has a softer, more cohesive look. Which looks less artificial and more in line with real-world lighting behaviour.

Thin Objects

Thin objects like leaves and intricate plant details are handled exceptionally well in the first image. Unlike traditional SSAO, which often over-darkens thin geometry due to its lack of geometric awareness, the first image uses a heuristic to account for thin objects.

This approach prevents excessive shadowing, allowing the finer details, like the leaves and thin furniture components, to remain visible and realistic. The result is a more convincing representation of vegetation and in general thin objects.

Large Occluders

This is the biggest difference. Occlusion from large objects such as trees and roofs are still a problem. This is because our hemisphere is not big enough. We could solve this problem by overlaying outputs from small and big hemispheres. Or just use raytracing to have a better global illumination.

Contrast

The overall contrast in the first image is pretty smooth, it's still way more washed out than in the second image but it's better. The smooth contrast makes the lighting transitions feel more natural, contributing to a cleaner, more polished visual style.

7 Merged Technique

I'm going to explain how I tried to combine two different techniques to create something that gives the best of both worlds.

My goal is to find a middle ground between the two methods. In the end, I want a technique that's faster than the GTAO method but still looks better than the Fast SSAO method.

7.1 Implementation Details

The merging process is actually pretty simple. I decided to take the Fast SSAO technique as the base but replace its SSAO calculation step with the one used in GTAO.

This way, I'm keeping the speed of the Fast SSAO method while improving the visual quality by using GTAO for the key calculations.

Here's how I implemented the merged technique step by step:

- Downscale the Depth Buffer to quarter size into a R16_Float texture.
- Calculate GTAO with the downscaled depth buffer.
- Perform the two depth aware gaussian blur passes.
- Upsample the final blurred image.

For the sampling, I stick with the same count as in the GTAO technique, which uses 2×8 samples. This ensures that the quality remains consistent with GTAO, while the overall process is optimized for better speed.

7.2 Results

Performance

Pass	Average Time in milliseconds
<i>Downsample Depth Pass</i>	0.0390 ms
<i>Compute GTAO Pass</i>	0.3015 ms
<i>Blur GTAO Pass</i>	0.1137 ms
<i>Upsample GTAO Pass</i>	0.0942 ms
<i>Full GTAO Calculations:</i>	0.5486 ms

Table 5: Average timings for the merged technique.

Each step of the process was measured, and the overall average time for all the calculations came out to approximately 0.5486 milliseconds per frame, which is just a little bit under 0.6 milliseconds in total. When compared to the "fast" SSAO technique, this method is roughly 0.1 milliseconds slower, which, considering the improvements, is still a really impressive and positive result overall.

the merged technique averages about 33.4% slower than the "fast" SSAO method. But it's also 66.3% faster than the full GTAO technique, which is a huge improvement in speed.

In my opinion, this is a great trade-off. The merged technique keeps the performance close to the fast technique. Even a small improvement in how the image looks would justify using this method instead of the fast technique, especially for games or applications where both speed and quality matter.

Visual

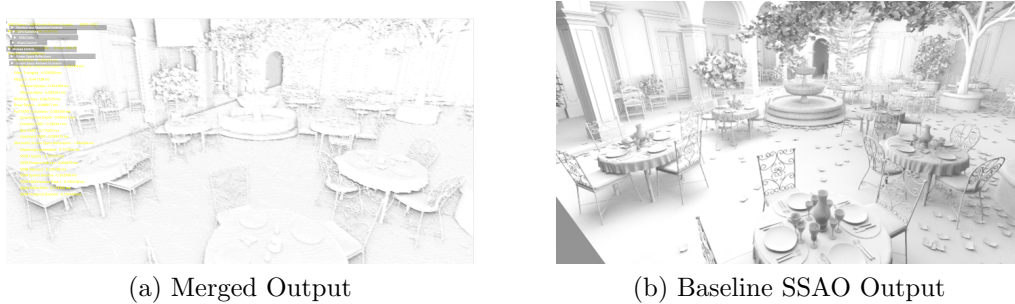


Figure 11: Comparison of the Merged Technique

Edges

When comparing the edges, we did lose some of the sharpness and detail, but they are still noticeable and clear enough. The biggest difference is that the further away you go, the more detail is lost compared to the baseline. However, the edges in this implementation are not as harsh or abrupt as they are in the fast SSAO implementation, which helps make the overall image look smoother and more natural.

Thin Objects

For thin objects, like small items or leaves, there isn't any noticeable over-darkening, which is great. The only time it becomes an issue is when these objects are grouped or clustered too closely together, as this can make them slightly darker than they should be. This isn't a big problem, though, since we could adjust the heuristic used to fix it if needed, making this a pretty minor drawback overall.

Large Occluders

Large occluders remain a weak spot across all the techniques, but this implementation does better than the fast technique while still falling short compared to the GTAO implementation. It's an area that could still use some improvement, but for now, it's a good middle ground between speed and quality.

Contrast

The overall contrast in the merged output looks fairly smooth and balanced. It's harsher than the baseline and more similar to the fast technique in terms of sharpness, but it's still noticeably better.

Discussion

The theoretical framework provided a robust foundation for understanding Screen Space Ambient Occlusion (SSAO), its workflow, and its various enhancements/improvements over the years. It highlighted the importance of extracting depth and normal buffers, employing efficient sampling techniques, and integrating ambient occlusion seamlessly into the lighting pass for optimal realism. This served as a reference point for evaluating the results of my implementation.

The implementation emphasized the necessity of balancing visual fidelity and computational efficiency, particularly for real-time rendering. Leveraging compute shaders allowed for some minor performance optimizations by bypassing bottlenecks and utilizing direct memory access.

However, some challenges highlighted in the theoretical framework remained evident in the results. For example, while SSAO effectively simulates soft shadowing, its reliance on screen-space sampling introduces inaccuracies, particularly for occlusions caused by large objects outside the visible camera view. This aligns with the framework's assertion that the limited hemisphere size affects occlusion accuracy for broader terrains.

Overall, the results validate the theoretical framework while also highlighting areas for further refinement. The use of compute shaders and heuristic-based SSAO approximations has shown that real-time ambient occlusion techniques can be combined to get a technique in the middle ground.

Conclusion

The results confirmed what i expected: Ground Truth Ambient Occlusion (GTAO) is much more accurate, but it is also much slower. While GTAO produces very detailed and natural-looking results, its high performance cost makes makes it not the best solution for everything.

In this project, i successfully combined ideas from both GTAO and the fast Screen Space Ambient Occlusion (SSAO) to create a combined method that balances accuracy and performance. This new approach sits between the two: it is much faster than pure GTAO while keeping more detail than the fast SSAO. By using passes that speed up the calculations inspired by the fast SSAO, the new technique works well in real-time.

However, this faster method does come with some trade-offs. While it is quicker, it doesn't capture all the small details and subtle effects that pure GTAO does. For example, shadows in tight spaces or complex shapes aren't as sharp or realistic. Even so, the results are still good enough for real-time use and look convincing.

In conclusion, the new method is a good mix of speed and quality. It shows that by combining the strengths of both techniques, we can create a solution that works well for games and other real-time applications. In the future, we could try to recover some of the missing details or explore other improvements to make the method even better.

Future work

The merged technique could further be improved by common used tricks on Screen Space Ambient Occlusion.

- We could use a temporal reprojection technique where we could early out if the earlier frames have no occlusion.
- Right now the GTAO is still fairly simple. The paper also goes over multiple bounces. This could greatly improve the visuals of the generated ambient occlusion.
- I would like to rework the upsampler as i don't like how the interpolation between pixels is at the moment.
- If we decide to not use a temporal solution i would move the blurring after the upscaling to improve the blurring. I am not really happy with the visual result of the blurring as it is now. I would even skip out the blurring if i decide to go with a temporal solution because that will automatically smear out the occlusion.
- Right now i use the normals from the normal map, these are high fidelity. This could cause flickering and artifacts while using a down-sampled buffer. Instead you could reconstruct the normals from the depth buffer. This would also improve the bandwidth as you only need 1 bound texture to the pipeline.
- Increasing the radius gives a better and smoother Screen Space Ambient Occlusion, This comes with a performance penalty, as our samples are then further away and are no longer in the texture cache. This latency can be minimized by using a mipmap chain of the depth buffer. The further the sample from the pixel the smaller the mip will be that we use.

Critical Reflection

Before starting this thesis, I only knew about a slow and not-so-good-looking Screen Space Ambient Occlusion (SSAO) technique. I found it interesting and wanted to learn more about it, which is what inspired me to focus on it for this project.

While working on the thesis, I got to talk to some really interesting and experienced people. They taught me so much, not just about SSAO but also about other important things like how to debug shaders better and how the math is connected. I also learned more about radiology and how it connects to computer graphics.

This project wasn't just about improving my technical skills. It also helped me grow personally. I learned how to break down tough problems, stay organized, and not give up even when things got hard. Seeing my work improve step by step was really satisfying and made me even more excited about graphics programming.

Looking back, I'm proud of how much I've learned and how far I've come. This project taught me skills I'll use in the future and gave me more confidence to take on challenges. It's been a great experience that I'll always remember.

References

- [1] F. P. Aalund, “A Comparative Study of Screen-Space Ambient Occlusion Methods Bachelor Thesis.”
- [2] F. Bauer, M. Knuth, A. Kuijper, and J. Bender, “Screen-Space Ambient Occlusion Using A-Buffer Techniques,” in 2013 International Conference on Computer-Aided Design and Computer Graphics, Guangzhou, China: IEEE, Nov. 2013, pp. 140–147. doi: 10.1109/CADGraphics.2013.26.
- [3] J. Chapman, “SSAO tutorial.” [Online]. Available: <https://john-chapman-graphics.blogspot.com/2013/01/ssao-tutorial.html>
- [4] R. Dimitrov, L. Bavoil, and M. Sainz, “Horizon-split ambient occlusion,” in Proceedings of the 2008 symposium on Interactive 3D graphics and games, Redwood City California: ACM, Feb. 2008, pp. 1–1. doi: 10.1145/1342250.1357017.
- [5] W. Engel, Ed., ShaderX7: advanced rendering techniques. in ShaderX series. Boston, Mass: Charles River Media, 2009.
- [6] T.-D. Hoang and K.-L. Low, “Efficient screen-space approach to high-quality multiscale ambient occlusion,” *Vis Comput*, vol. 28, no. 3, pp. 289–304, Mar. 2012, doi: 10.1007/s00371-011-0639-y.
- [7] J. Jimenez, X.-C. Wu, A. Pesce, and A. Jarabo, “Practical Real-Time Strategies for Accurate Indirect Occlusion.”
- [8] D. Lenaerts, “An alternative implementation for HBAO.” [Online]. Available: <https://www.derschmale.com/2013/12/20/an-alternative>

-implementation-for-hbao-2/

[9] J. M. Méndez, “Simple SSAO.” [Online]. Available: https://www.gamedev.net/tutorials/_/technical/graphics-programming-and-theory/a-simple-and-practical-approach-to-ssao-r2753/

[10] M. Mittring, “Finding next gen: CryEngine 2,” in ACM SIGGRAPH 2007 courses, San Diego California: ACM, Aug. 2007, pp. 97–121. doi: 10.1145/1281500.1281671.

[11] I. Quilez, “screen space ambient occlusion.” [Online]. Available: <https://iquilezles.org/articles/ssao/>

[12] T. Ritschel, T. Grosch, and H.-P. Seidel, “Approximating dynamic global illumination in image space,” in Proceedings of the 2009 symposium on Interactive 3D graphics and games, Boston Massachusetts: ACM, Feb. 2009, pp. 75–82. doi: 10.1145/1507149.1507161.

[13] A. Tardif, “SSAO.” [Online]. Available: <https://alextardif.com/SSAO.html>

[14] S. Wojciech, “Robust Screen Space Ambient Occlusion in 1 ms in 1080p on PS4,” in Gpu Pro, vol. 1, 3 vols., in Gpu Pro, no. 1, vol. 1. Black Cat.

[15] A. E. M. Zerari and M. C. Babahenini, “Screen Space Ambient Occlusion Based Multiple Importance Sampling for Real-Time Rendering,” 3D Res, vol. 9, no. 1, p. 1, Mar. 2018, doi: 10.1007/s13319-017-0152-9.

[16] “3D Game Shaders For Beginners,” 3D Game Shaders For Beginners. [Online]. Available: <https://leltier.github.io/3d-game-shaders-for-beginners/ssao.html>

[17] “Graphics Compendium SSAO.” [Online]. Available: <https://graphicscompendium.com/gamedev/17-ssao>

[18] “Screen Space Ambient Occlusion.” [Online]. Available: https://en.wikipedia.org/wiki/Screen_space_ambient_occlusion

[19] “Speed of ComputeShader vs. PixelShader.” [Online]. Available: <https://stackoverflow.com/questions/5195214/speed-of-computeshader-vs-pixelshader>

[20] “When is a compute shader more efficient than a pixel shader for image filtering?” [Online]. Available: <https://computergraphics.stackexchange.com/questions/54/when-is-a-compute-shader-more-efficient-than-a-pixel-shader-for-image-filtering>